



Hiring compliance, audited by a **team of AI agents**.

Four specialized agents collaborate inside a **Band room** to flag EEOC and pay-transparency risk — then hand off, score, and sign a defensible, cited audit memo.

Band of Agents Hackathon

Track 3 — Regulated & High-Stakes

Main Prize · **Best Use of AI/ML API**

— WHY THIS MATTERS

Hiring language is a legal minefield — and nobody reviews every posting.

A single job post can violate federal anti-discrimination law and a patchwork of state pay-transparency statutes at once. Review is manual, slow, and inconsistent.

\$440M+

EEOC monetary recovery from employers in a single year

5+ states

Pay-transparency laws with distinct posting rules (CA, NY, CO, WA, MA)

Every req

Needs an audit — but legal can't review them all by hand

— ONE COMMAND, ONE AUDIT

Feed in a hiring packet. Get back a defensible memo.

INPUT — THE HIRING PACKET

- Job posting copy
- Compensation band
- Interview scorecard
- Company size & location

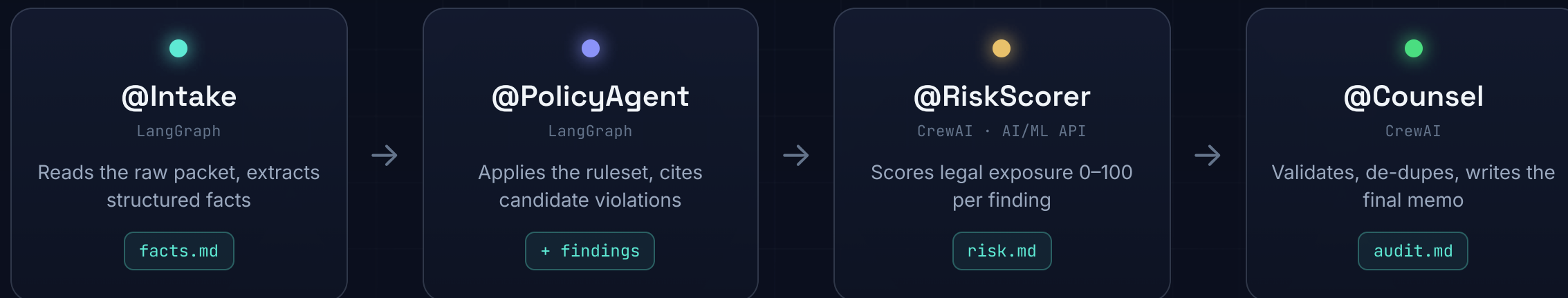


OUTPUT — AUDIT.MD

- Findings by severity
- Real statute citations
- Quoted evidence
- Human sign-off gate

— FOUR AGENTS, TWO FRAMEWORKS, ONE ROOM

Each agent owns one job and hands off the baton.



Chat is for coordination. Files are for content.

- **Agents talk via @mentions**

Each agent posts a short handoff in the room and @mentions the next. Only the mentioned agent wakes up — routing is explicit and auditable.

- **Content lives in shared notes**

facts.md, risk.md and audit.md sit in a shared workspace. Chat stays lightweight; the artifacts carry the substance.

- **Cross-framework, one protocol**

LangGraph and CrewAI agents interoperate seamlessly — Band is the common bus between them.

band room · #hireguard-audit

@Intake facts.md ready — 1 packet extracted.
→ mentions @PolicyAgent

@PolicyAgent 6 candidate findings posted.
→ mentions @RiskScorer

@RiskScorer risk.md ready — verdict HIGH.
→ mentions @Counsel

@Counsel audit complete — awaiting
human sign-off.

No finding gets a verdict until the AI/ML API returns.

@RiskScorer sends every candidate finding to the **AI/ML API**, which weighs three dimensions and returns a structured exposure score.

- **Severity**

Statutory penalty & litigation exposure

- **Likelihood**

How clearly the evidence establishes a violation

- **Jurisdiction**

Whether the rule actually attaches here

scoreexposure() → AI/ML API

```
# PAYTRANS-CA-SB1162
{
  "exposure_score": 88,
  "severity": "severe",
  "likelihood": "high",
  "jurisdiction_attaches": true,
  "rationale": "CA employer,
    15+ staff, no posted range"
}
```

— AGENTS CHECK EACH OTHER

@Counsel can bounce a weak finding back to @PolicyAgent.

Before signing, @Counsel adversarially re-examines every Critical finding. If the evidence is thin or the jurisdiction doesn't clearly attach, it sends the finding **back into the room** for re-examination — a visible, defensible review loop rather than a single-pass rubber stamp.

band room · the re-loop

```
@Counsel @PolicyAgent please re-examine EEOC-CRIM-HISTORY –  
  evidence is thin, no blanket exclusion in the posting.  
@PolicyAgent agreed – downgrading to Suggestion. Re-scored.  
@Counsel finalized. Writing audit.md.
```

— AUDIT.MD — GENERATED END-TO-END

The Acme demo packet → HIGH, six cited findings.

HireGuard Compliance Audit — Acme Robotics, SE role

VERDICT: HIGH

CRITICAL	PAYHIST-SALARY-BAN	Posting asks for the candidate's current salary
CRITICAL	PAYTRANS-CA-SB1162	CA employer, 15+ staff, no posted salary range
RISK	EEOC-ADEA-AGE	"young, energetic, recent grad" age-proxy language
RISK	EEOC-NATL-ORIGIN	Requires a "native English speaker"
RISK	EEOC-ADA-DISABILITY	"must lift 40 lbs" with no accommodation note
GAP	EEOC-DISPARATE-SUBJECTIVE	"culture fit" scorecard criterion, unanchored



Four agents. One defensible audit.

A real, multi-framework agent team collaborating on Band — with the AI/ML API at the load-bearing core of every risk verdict.

Band SDK · room + @mention handoffs

AI/ML API · exposure scoring

LangGraph · Intake + Policy

CrewAI · Risk + Counsel

Python · uv · Pydantic